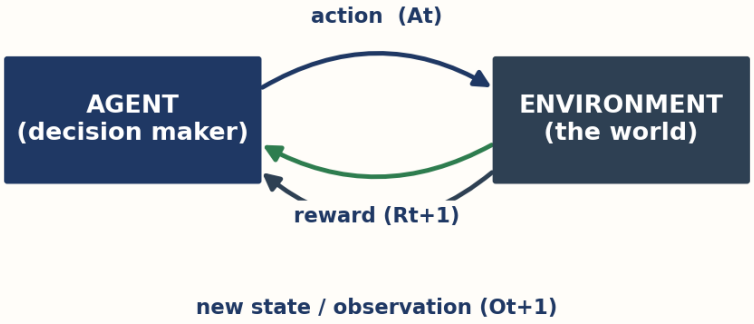
QWIK GUIDE

Reinforcement Learning: Essentials

A beginner-friendly introduction to agents, environments, rewards, and how machines learn by doing



The agent acts on the environment; the environment answers with a reward and a new state.

The reinforcement learning loop at a glance.

1. What Is Reinforcement Learning?

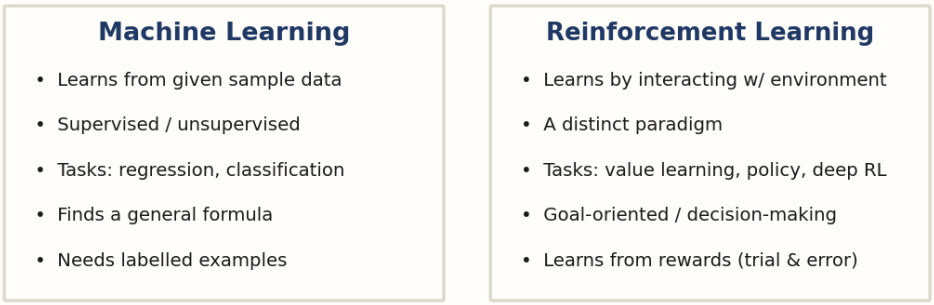
Imagine teaching a dog a new trick. You don't hand it a rulebook. Instead, the dog tries something, and you give it a treat when it does well. Over time, the dog learns which actions earn treats. **Reinforcement learning (RL)** teaches software the very same way: a program tries actions in some world, receives rewards or penalties, and gradually learns the behaviour that earns the most reward.

RL is often called the **science of decision-making**. It is **goal-oriented**: the learner is always working toward a goal, improving its performance over time through trial and error. A useful way to say it is that RL programs learn to identify the best actions to take in each situation.

> **KEY IDEA**
>
> RL is commonly treated as a distinct machine-learning paradigm because learning is driven by **interaction and reward**, rather than only labelled examples (supervised) or unlabelled data (unsupervised). In practice, RL can still borrow from those paradigms — for example using supervised or imitation learning, or offline datasets.

Reinforcement Learning vs. Machine Learning

Ordinary machine learning (ML) learns from a fixed pile of example data using supervised or unsupervised algorithms such as regression and classification. RL is different: it has no fixed dataset. Instead it **interacts** with an environment and collects information as it goes.



Machine learning learns from given data; RL learns by interacting.

Aspect	Machine Learning	Reinforcement Learning
Data source	Given / existing sample data	Collected by interacting with an environment
Algorithms	Supervised & unsupervised	A distinct paradigm (may borrow from both)
Typical tasks	Regression, classification	Value learning, policy learning, deep RL
Goal	Find a general formula	Make good decisions to maximize reward

GOOD TO KNOW

When RL is combined with a deep neural network, it is called **deep reinforcement learning**. The agent — implemented partly by the neural network — interacts with the environment, and its behaviour can be guided by a policy.

2. The Essential Elements

Every RL system is built from the same handful of parts. Get comfortable with these words and the rest of the subject falls into place.

The essential building blocks of reinforcement learning.

Element	Plain-language meaning
Agent	The decision maker — the program that takes actions.
Action	A move the agent can make in the environment.
Environment	The world; it takes the agent's action and returns a reward and the next state.
State	The current situation of the environment.
Observation	What the agent actually receives — may equal the state, or be only partial/noisy information about it.
Reward	Immediate feedback (a single number) measuring how good or bad the last action was.
Return	The discounted sum of future rewards — what value functions estimate.
Policy	The agent's strategy: which action (or distribution over actions) to choose in each state.
Value	The expected return from a state, with discounting.
Discount factor	How much future rewards are shrunk compared with rewards now (often written gamma).

3. How the Pieces Talk: The RL Flow

Reinforcement learning is an interconnected loop. The agent and the environment share a communication channel. The agent reads an **observation** of the current state, decides on an **action**, and sends it to the environment. The environment responds with a **reward** and the next state. That reward-for-action exchange, repeated over and over, is the whole game.

NOTATION YOU'LL SEE

R_t = reward at time step t • O_t = observation the agent receives • A_t = action taken • R_{t+1} / O_{t+1} = reward and observation the environment emits next.

A **state transition** is simply the change from one state to another. It can happen over time, or as a direct result of an action the agent took. Keep two ideas separate: the **state** is the environment's true situation, while an **observation** is what the agent receives — these are the same only when the environment is fully observable.

Reward, Return, and the Reward Hypothesis

Everything in RL rests on one bold idea, the **reward hypothesis**: *all goals can be described as the maximization of expected cumulative reward.* Whatever we want the agent to achieve, we express it as a reward signal, and the agent's job is to collect as much reward as possible over the long run.

It helps to separate two terms. A **reward** (R_{t+1}) is the single number the environment gives after one action. The **return** is the discounted sum of all future rewards from a point onward. Value functions estimate expected *return*, not merely one step's reward. A related learning signal is the **temporal-difference (TD) error**: the difference between the agent's current value estimate and a target built from the received reward plus its discounted estimate of what comes next. That gap is what drives learning — and you'll see it in the code later.

4. What an Agent Actually Does

At each moment, an agent follows three simple steps. This is the perceive–decide–act cycle that repeats throughout an episode.

The three steps an agent takes to make a decision.

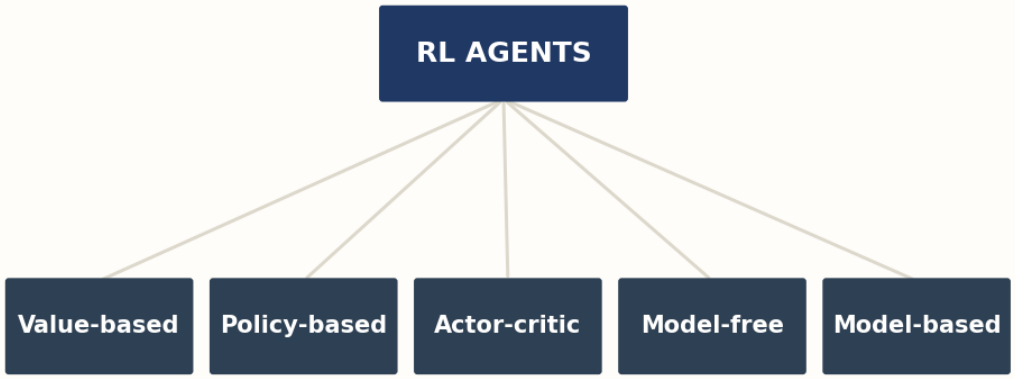
1. **Perceive the environment** — collect the information needed to choose an action.
2. **Take a decision** — use the current observation (and its policy) to choose.
3. **Convert the decision into an action** — carry it out in the environment.

ROBOT EXAMPLE

For a walking robot, the environment is the space it moves in, the robot is the agent, and observations might be its joint angles. Each body movement earns a reward and can trigger the next action.

5. Ways to Classify Agents

These are common, **overlapping** ways to describe RL methods — not mutually exclusive boxes. A single method usually sits in several of them at once.



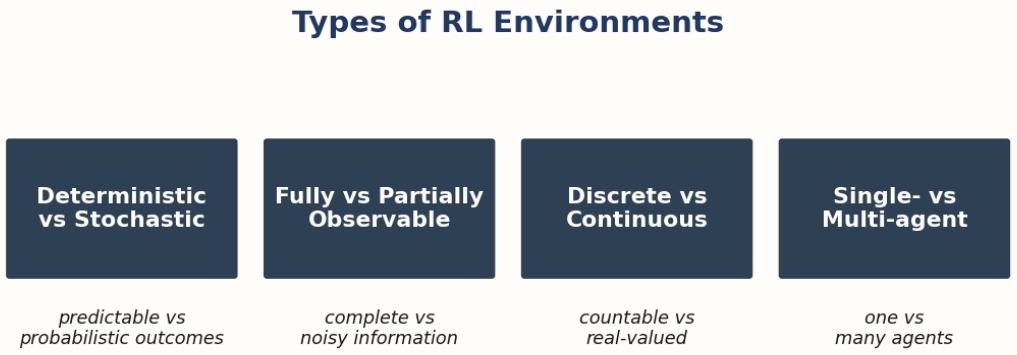
Common, overlapping ways to classify RL methods.

Classification	What it describes
Value-based	Behaviour comes from learned **value** estimates; the policy is derived from them.
Policy-based	Learns a policy directly; strong in large / continuous action spaces.
Actor-critic	**Combines** policy learning (actor) and value learning (critic); can be model-free or model-based.
Model-free	Learns from experience only, with no learned model of the environment's dynamics.
Model-based	Also uses a model of the environment (a known or learned dynamics/reward model).

> **THEY OVERLAP**
>
> For example, actor-critic methods are typically *both* policy- and value-based, and may be either model-free or model-based. 'Value-based vs policy-based' and 'model-free vs model-based' are two **different questions** you can ask about the same method.

6. Types of Environments

Environments are classified along a few dimensions. Knowing which kind you face tells you how hard the problem is.



Prominent categories of reinforcement learning environments.

Dimension	Meaning
Deterministic	A given state and action lead to a single, predictable next state and reward.
Stochastic (non-deterministic)	Transitions or rewards are probabilistic; the same action can lead to different outcomes.
Fully observable	The agent receives enough information to determine the relevant state.
Partially observable	The agent receives incomplete or noisy information, not the full state.
Discrete / Continuous	States and actions are countable, or real-valued ranges.
Single-agent	One agent interacting with one environment.
Multi-agent	Several agents, each with its own goals, across changing environments.

7. Getting Started with the Tools

OpenAI Gym was the original toolkit for developing and comparing RL algorithms, and much older material still refers to it (often paired with a now-discontinued platform called OpenAI Universe). Today the maintained path is simpler and more capable.

> **USE MAINTAINED LIBRARIES**
>
> The original **gym** package is no longer maintained, and **OpenAI Universe was discontinued years ago** — don't start with either. Use the modern replacements below.

Modern tooling

Gymnasium provides a maintained interface for single-agent RL environments, including FrozenLake (`import gymnasium as gym`). For ready-made algorithm implementations, consider **Stable-Baselines3**. For multi-agent environments, consider **PettingZoo**. Install only the libraries your chosen environment and algorithm actually need.

Because `pip install gymnasium` alone may not pull in every environment's optional dependencies, install the extra for the environment family you want — here, the **toy-text** family that contains FrozenLake:

```
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
python -m pip install --upgrade pip
pip install "gymnasium[toy-text]" numpy matplotlib
```

8. Hands-On: See RL in Code

The companion **rl_samples.zip** contains four small, heavily commented Python scripts. They all learn on the **same** tiny world — **FrozenLake**, a 4×4 grid where the agent must cross the ice from Start (S) to Goal (G) without falling in a Hole (H). Using one shared environment keeps your attention on the ideas, not the setup.

> **THE SHARED WORLD**
>
> 16 states (tiles 0–15) • 4 actions (Left, Down, Right, Up) • reward +1 only for reaching the goal. Every script imports it from `shared_env.py`. On this small map the tile number fully identifies the state, so observation = state here.

The environment in one function

This helper builds the same world for every demo. We set `is_slippery=False` so the ice is **deterministic** — the easiest case to reason about. Setting it to `True` makes the world **stochastic**, where the same action can slip to different tiles.

```
import gymnasium as gym

def make_env(render_mode=None, slippery=False):
    return gym.make('FrozenLake-v1', map_name='4x4',
                    is_slippery=slippery, render_mode=render_mode)
```

Exploration vs. exploitation (epsilon-greedy)

Before it can exploit good moves, an agent must **explore** to discover them. The demo uses a simple **epsilon-greedy** rule: with probability epsilon, try a random action (explore); otherwise take the best action known so far (exploit). Epsilon starts high and shrinks as the agent grows confident.

```
if rng.random() < epsilon:
    action = env.action_space.sample()      # explore: try anything
else:
    action = int(np.argmax(Q[state]))       # exploit: best known move
```

The learning step (Q-learning)

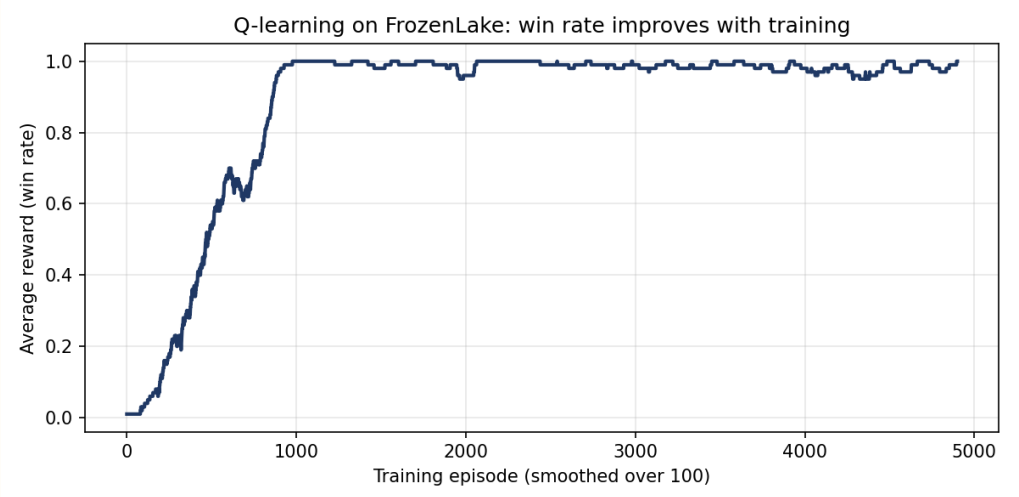
Script 02 trains a **value-based, model-free** agent. The heart of it is the update below. Note the terminal-state handling — when an episode ends there is no next state to look ahead to, so we do *not* bootstrap. The bracketed quantity is the **TD error**:

```
if terminated:
    target = reward                         # no next state to look ahead to
else:
    best_next = np.max(Q[next_state])
    target = reward + gamma * best_next     # gamma = discount factor

td_error = target - Q[state, action]        # temporal-difference error
Q[state, action] += alpha * td_error        # nudge estimate toward target
```

Written as one line, this is the classic tabular Q-learning update: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$, where the bracketed term is the TD error (and for a terminal step the target is just r).

On the deterministic (`is_slippery=False`) FrozenLake map, a well-tuned tabular Q-learning agent can reach near-perfect performance after sufficient training. The learning curve below — from script 04 — shows the win rate climbing as the value estimates improve.



Illustrative run: win rate rising over training episodes (deterministic FrozenLake; seed 0, 5000 episodes, alpha 0.8, gamma 0.95, 100-episode smoothing). Exact numbers vary with seed and hyperparameters.

Running the samples

Full step-by-step setup is in the zip's `README.txt`. In short:

```
python -m venv .venv
source .venv/bin/activate     # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python run_all.py             # runs all four demos in order
```

RUN THEM YOUR WAY

`run_all.py` runs everything at once, or you can run each script one at a time to study a single idea. Just run script 02 before 03 and 04, since it creates the files they use.

9. Vocabulary & Concept List

Every term and concept from this guide, gathered for quick reference and listed alphabetically.

Action

The set of all possible moves an agent can make; a task the agent performs in the environment.

Actor-Critic Method

A family of techniques that combines policy learning (the 'actor') with value learning (the 'critic'), based on the policy-gradient theorem; can be model-free or model-based.

Agent

The decision maker; the program that observes the environment and takes actions.

Agent Steps

The decision cycle: perceive the environment, take a decision, convert the decision into an action.

Deep Reinforcement Learning

RL combined with a deep neural network; the agent, implemented partly by the network, interacts with the environment and can be guided by a policy.

Deterministic Environment

One where a given state and action produce a single, predictable next state and reward.

Discount Factor

A factor (often gamma) multiplied by future rewards to reduce their effect on the agent's current choice.

Discrete or Continuous Environment

Classifies environments by whether their states/actions are countable or real-valued.

Environment

The surroundings that take the agent's action (and current state) as input and return a reward and the next state.

Exploration vs. Exploitation

The trade-off between trying new actions to gather information (explore) and using the best known action to earn reward (exploit); epsilon-greedy is a simple balance.

Fully Observable Environment

One where the agent receives enough information to determine the relevant state (observation identifies the state).

Machine Learning (ML)

Deriving a model from existing sample data using supervised or unsupervised algorithms (e.g. regression, classification). Contrasts with RL, which learns by interacting.

Model-Based Agent

Learns or uses a model of the environment (a dynamics/transition and reward model) in addition to a policy or value function.

Model-Free Agent

Learns a policy or value function from experience only, without a learned model of the environment's dynamics.

Multi-Agent Environment

Multiple agents, each with their own goals and knowledge, dynamically interacting with more than one environment.

Observation

The information the agent actually receives (O_t); equals the state only when the environment is fully observable, otherwise partial or noisy.

OpenAI Gym

The original toolkit for developing and comparing RL algorithms; now unmaintained. Its maintained successor is Gymnasium.

OpenAI Universe

A now-discontinued platform for training AI across many programs; historical only, not used in modern RL.

PettingZoo

A maintained library of multi-agent RL environments.

Policy

A rule specifying what action — or distribution over actions — the agent chooses in each state or observation.

Policy-Based Agent

An agent that learns a policy directly; converges well and suits high-dimensional or continuous action spaces.

Q-learning

A value-based, model-free method that learns a table of state-action values (Q-values) to derive a good policy.

Reinforcement Learning (RL)

A distinct machine-learning paradigm in which an agent learns from interaction and reward in a known or unknown environment; the goal-oriented science of decision-making.

Return

The discounted sum of future rewards from a point onward; value functions estimate expected return.

Reward

Immediate feedback measuring the success or failure of an action; a scalar signal (R_t) used to train agents.

Reward Hypothesis

The foundational idea that all goals can be described as maximizing expected cumulative reward.

Single-Agent Environment

One agent with its own goals, actions, and knowledge, interacting with one environment.

Stable-Baselines3

A maintained library of ready-made RL algorithm implementations.

State

A concrete, immediate situation of the environment; used to decide how state transitions occur.

State Transition

The change from one state to another, over time or as a result of an action.

Stochastic (Non-Deterministic) Environment

One where transitions or rewards are probabilistic, so the same action can lead to different outcomes.

Temporal-Difference (TD) Error

The difference between the agent's current value estimate and a target built from the received reward plus its discounted estimate of the next state's value; the core learning signal in TD methods.

Value

The expected return from a state (or state-action pair), with discounting applied.

Value-Based Agent

An agent whose behaviour is derived from learned value estimates.

10. Where to Learn More

These beginner-friendly resources go deeper than this guide. A suggested path: start hands-on with Hugging Face, watch David Silver for the theory, then use OpenAI Spinning Up while you train your own agents. The first two are stable foundational references; the tutorials and courses may be updated over time, so check for the latest version.

David Silver's RL Course (UCL / DeepMind) — foundational
https://www.youtube.com/playlist?list=PLqYmG7hTraZDM-OYHWgPebj2MfCFzFObQ
A 10-lecture series giving an intuitive mathematical introduction to MDPs, dynamic programming, and policy gradients.

OpenAI Spinning Up in Deep RL — foundational
https://spinningup.openai.com/
OpenAI's educational resource for key RL algorithms, terminology, and PyTorch implementations.

Hugging Face Deep RL Course — may be updated
https://huggingface.co/deep-rl-course
Free, hands-on course from Q-learning to PPO using Colab notebooks and Stable-Baselines3.

Lilian Weng: A (Long) Peek into RL — may be updated
https://lilianweng.github.io/posts/2018-02-19-rl-overview/
A high-quality technical summary bridging classic RL and modern deep RL (DQN, actor-critic, policy gradients).

DataCamp: An Introduction to Q-Learning — may be updated
https://www.datacamp.com/tutorial/introduction-q-learning-python-tutorial
A code-first tutorial building Q-learning from scratch, covering Q-tables and exploration vs. exploitation.

ABOUT THE SAMPLE CODE

The four scripts in `rl_samples.zip` (explore, train, watch, plot) all share one FrozenLake environment. See the included `README.txt` for virtual-environment setup, `requirements.txt`, and how to run them together or one at a time.